

Functions

Outline



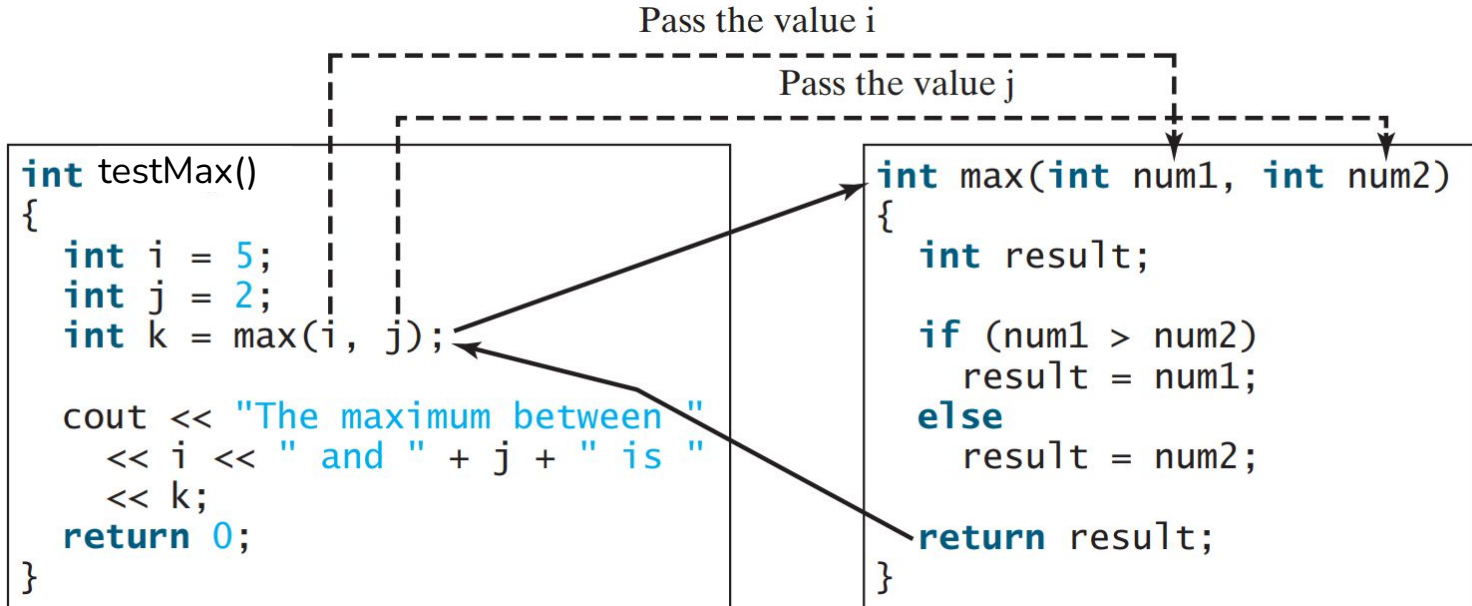
- **Calling a Function**
- **void Functions**
- **Passing Arguments by Value**
- **Modularizing Code**
- **Overloading Functions**
- **Function Prototypes**

Outline



- **Inline Functions**
- **Default Arguments**
- **Local, Global, and Static Local Variables**
 - Variables in a for Loop
 - Static Local Variables
- **Passing Arguments by Reference**
- **Constant Reference Parameters**

A function **must be defined before it is invoked**. Since the max function is invoked by the main function, it must be defined before the main function



When the max function is invoked, variable `i`'s value **5** is passed to `num1`, and variable `j`'s value **2** is passed to `num2` in the max function. When the return statement in the `max` function is executed, the `max` function **returns the control to its caller**.

void Functions

A void function **does not return a value.**

A return statement is **not needed** for a void function, **but it can be used for terminating the function and returning control to the function's caller.** The syntax is simply

```
return;
```

Occasionally the **exit(int)** function in the **cstdlib** header, can be used to terminate the program from the function immediately. For example:

```
exit(1);
```

TestVoidFunction.cpp

Passing Arguments by Value

By default, the arguments are passed by value to parameters when invoking a function. **The power of a function is its ability to work with parameters.**

When calling a function, the arguments **must be given in the same order as their respective parameters in the function signature.** This is known as **parameter order association.** For example:

```
void nPrint(char ch, int n)
{
    for (int i = 0; i < n; i++)
        cout << ch;
}
```

```
nPrint('a', 3);
```

VS

```
nPrint(3, 'a');
```

Modularizing Code



Modularizing makes the code easy to maintain and debug and enables the code to be reused. Functions can be used to reduce redundant code and enable code reuse. **Functions can also be used to modularize code and improve the program's quality.**

TestVoidFunction.cpp

Overloading Functions



Overloading functions are used to define the **functions with the same name as long as their signatures are different.**

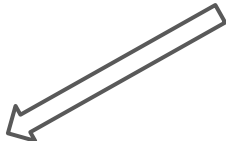
- Overloaded functions **must have different parameter lists.**
- Functions cannot be overloaded based on different return types.

The function **name** and the **parameter** list **together** constitute the **function signature.**

TestFunctionOverloading.cpp

Ambiguous invocation causes a compile error. Consider the following code:

```
#include <iostream>
using namespace std;
int maxNumber(int num1, double num2)
{
    if (num1 > num2)
        return num1;
    else
        return num2;
}
double maxNumber(double num1, int num2)
{
    if (num1 > num2)
        return num1;
    else
        return num2;
}
int main()
{
    cout << maxNumber(1, 2) << endl;
    return 0;
}
```



Both `maxNumber(int, double)` and `maxNumber(double, int)` are possible candidates to match `maxNumber(1, 2)`.

Change `maxNumber(1, 2)` to `maxNumber(1, 2.0)` so that it will match the first `maxNumber` function to avoid a compile error.

Function Prototypes

A function prototype declares a function **without having to implement it**.

Before a function is called, its header must be declared.

- One way to ensure this is to **place the definition before all function calls**.
- Another approach is to **define a function prototype before the function is called**.

A function prototype, also known as function declaration, is a function header without implementation. The implementation is given later in the program. In the prototype, only the parameter types are needed, because C++ compiler ignores the parameter names.

TestFunctionPrototype.cpp

Functions can be declared with default argument values. The default values are passed to the parameters when a function is invoked without the arguments.

```
#include <iostream>
using namespace std;

// Display area of a circle
void printArea(double radius = 1)
{
    double area = radius * radius * 3.14159;
    cout << "area is " << area << endl;
}

int main()
{
    printArea();
    printArea(4);
    return 0;
}
```



This line invokes the function without passing an argument. In this case, the default value 1 is assigned to radius.

Implementing a program using functions makes the program easy to read and easy to maintain, but **function calls involve runtime overhead** (i.e., pushing arguments and CPU registers into the stack and transferring control to and from a function).

```
#include <iostream>
using namespace std;

inline void f(int month, int year)
{
    cout << "month is " << month << endl;
    cout << "year is " << year << endl;
}

int main()
{
    int month = 10, year = 2008;
    f(month, year); // Invoke inline function
    f(9, 2010); // Invoke inline function
    return 0;
}
```

C++ provides inline functions to avoid function calls. Inline functions are not called; rather, the compiler copies the function code in line at the point of each invocation.

Local, Global, and Static Local Variables

A variable must be **declared before it can be used**. The scope of a local variable starts **from its declaration and continues to the end of the block** that contains the variable.

- A variable defined inside a function is referred to as a local variable.
- Global variables are declared outside all functions and are accessible to all functions in their scope. The scope of a global variable starts from its declaration and continues to the end of the program.
- Local variables do not have default values, but global variables are defaulted to zero.

VariableScopeDemo.cpp

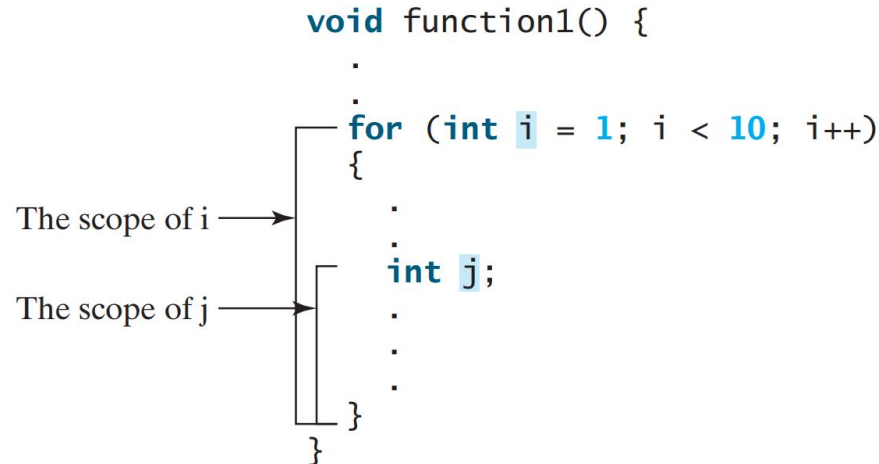
Variables in a for Loop

A variable declared in the initial-action part of a for-loop header has its scope in the entire loop. However, a variable declared inside a for-loop body has its scope limited in the loop body from its declaration to the end of the block that contains the variable, as shown below.

```
void function1() {  
    .  
    .  
    for (int i = 1; i < 10; i++)  
    {  
        .  
        .  
        int j;  
        .  
        .  
    }  
}
```

The scope of i →

The scope of j →



Variables in a for Loop

A variable declared in the initial-action part of a for-loop header has its scope in the entire loop. However, a variable declared inside a for-loop body has its scope limited in the loop body from its declaration to the end of the block that contains the variable, as shown below.

It is fine to declare `i` in two nonnesting blocks

```
void function1()
{
    int x = 1;
    int y = 1;

    for (int i = 1; i < 10; i++)
    {
        x += i;
    }

    for (int i = 1; i < 10; i++)
    {
        y += i;
    }
}
```

(a)

It is not a good practice to declare `i` in two nesting blocks

```
void function2()
{
    int i = 1;
    int sum = 0;

    for (int i = 1; i < 10; i++)
    {
        sum += i;
    }

    cout << i << endl;
    cout << sum << endl;
}
```

(b)

Static Local Variables



After a function completes its execution, all its local variables are destroyed. These variables are also known as automatic variables.

Static local variables are permanently allocated in the memory for the lifetime of the program. To declare a static variable, use the keyword **static**.

StaticVariableDemo.cpp

Passing Arguments by Reference

Parameters can be passed by reference, which makes **the formal parameter an alias of the actual argument**. Thus, changes, made to the parameters inside the function, were also made to the arguments.

```
#include <iostream>
using namespace std;

void maxValve(int value1, int value2, int max)
{
    if (value1 > value2)
        max = value1;
    else
        max = value2;
}

int main()
{
    int max = 0;
    maxValve(1, 2, max);
    cout << "max is " << max << endl;

    return 0;
}
```

(a)

```
#include <iostream>
using namespace std;

void maxValve(int value1, int value2, int& max)
{
    if (value1 > value2)
        max = value1;
    else
        max = value2;
}

int main()
{
    int max = 0;
    maxValve(1, 2, max);
    cout << "max is " << max << endl;

    return 0;
}
```

(b)

pass-by-reference

pass-by-value

If a program uses a pass-by-reference parameter and the parameter is not changed in the function, mark it constant to tell the compiler that the parameter should not be changed.

To do so, place the **const** keyword **before the parameter in the function declaration**. Such a parameter is known as constant reference parameter.

```
// Return the max between two numbers
int max(const int& num1, const int& num2)
{
    int result;
    if (num1 > num2)
        result = num1;
    else
        result = num2;
    return result;
}
```

The values of num1 and num2 cannot be changed in this function.

Reference



- **Revel for Introduction to C++ Programming and Data Structures by Daniel Liang, 5th edition, Pearson, 2022**